

Домашнє завдання 07 — Docker: образ, кеш, compose

Автор: Андрій Пономарьов

Середовище: macOS, Docker Engine 29.6.2 (Docker Desktop), Compose v5.3.1

Інструменти: `docker build`, `docker compose`, `curl`, `nc`; Claude Code — редактура

Завдання: (1) зібрати образ простого застосунку — ехес-форма CMD, окреме копіювання залежностей, свій `.dockerignore`; (2) виміряти ефект кешу — зібрати двічі, змінивши один символ у кодї, записати час обох збірок; (3) підняти зв'язку застосунок + база через compose — звернення за іменем сервіса, том для даних, healthcheck з умовою готовності; (4) зламати й полагодити — прибрати healthcheck і показати, що зміниться; опублікувати порт бази й пояснити, чому цього робити не варто.

Застосунок: **лічильник візитів** — Flask + PostgreSQL, ~30 рядків Python. Кожен GET / додає рядок у таблицю і повертає загальну кількість візитів.

Коротко про результат

Пункт	Результат
Образ	<code>python:3.12-slim</code> , 5 шарів, 173 МБ; ехес-форма CMD; контекст збірки — 1.12 кБ завдяки <code>.dockerignore</code>
Кеш	1-ша збірка 7.66 с (з них <code>pip install</code> 5.3 с) → 2-га після зміни одного символу 1.09 с
Compose	app ходить у <code>db</code> за іменем сервіса; том <code>db-data</code> переживає <code>down / up</code> ; <code>pg_isready - healthcheck + condition: service_healthy</code>
Зламали	без healthcheck compose або відмовляється стартувати, або app падає з <code>Connection refused</code> , поки база робить <code>initdb</code>
Порт бази	опублікований 5432 доступний будь-кому з хоста/мережі повз застосунок — публікувати не варто

1. Образ простого застосунку

`app/Dockerfile`:

```
FROM python:3.12-slim

WORKDIR /app

# Залежності копіюються окремо від коду: поки requirements.txt не змінився,
# шар з pip install береться з кешу і зміна коду не тягне переустановку.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 8000

CMD ["python", "app.py"]
```

Ключові рішення:

- **Екзес-форма CMD** (["python", "app.py"]) — процес стає PID 1 без проміжного `sh -c`, тому сигнали (SIGTERM від `docker stop`) доходять до Python напряму, і контейнер зупиняється миттєво, а не чекає 10 с на SIGKILL.
- **Окреме копіювання залежностей** — `COPY requirements.txt` + `RUN pip install` йдуть до `COPY app.py`. Docker кешує шари за контрольною сумою вхідних файлів: зміна коду інвалідує лише останній шар, а не встановлення залежностей.
- **.dockerignore** — виключає `__pycache__`, `.venv`, `.git`, md-файли й сам Dockerfile. Контекст збірки — **1.12 кБ** (лише `app.py` і `requirements.txt`). Без нього будь-яке сміття в теці потрапляло б у контекст і даремно інвалідувало кеш `COPY`.

Підсумковий образ — 173 МБ: базовий `python:3.12-slim` (~145 МБ) + 28.3 МБ залежностей + 1 кБ коду (`docs/image-info.txt`).

2. Ефект кешу

Зібрав двічі; між збірками змінив рівно один символ у `app.py` (додав `!` у рядок привітання):

Збірка	Що змінилось	Час	Що робив Docker
1-ша	—	7.66 с	усі шари з нуля; сам <code>pip install</code> — 5.3 с

Збірка	Що змінилось	Час	Що робив Docker
2-га	1 символ у <code>app.py</code>	1.09 с	<code>WORKDIR</code> , <code>COPY requirements.txt</code> , <code>RUN pip install —</code> CACHED ; перевиконано лише <code>COPY app.py</code>

Повні логи: `docs/build-1.txt`, `docs/build-2.txt`.

Чому така різниця. Docker перевіряє кеш пошарово зверху вниз: шар перевикористовується, поки інструкція та її вхідні дані не змінились. `requirements.txt` не мінявся → шар із встановленням залежностей (найдорожчий) узято з кешу. Зміна `app.py` інвалідувала лише крихітний шар `COPY app.py` . (1 кБ, 0.0 с). Якби код і залежності копіювались одним `COPY . .` до `pip install`, один символ у коді запуслав би повну переустановку залежностей — у реальних проєктах це хвилини на кожну збірку.

3. Зв'язка app + db через compose

`compose.yaml` :

```

services:
  app:
    build: ./app
    ports:
      - "8007:8000"  # 8000 на хості зайнятий іншим проєктом
    environment:
      DB_HOST: db      # звернення до бази за іменем сервіса
      DB_NAME: visits
      DB_USER: app
      DB_PASSWORD: secret
    depends_on:
      db:
        condition: service_healthy  # app стартує лише після готовності бази

  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: visits
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
    volumes:
      - db-data:/var/lib/postgresql/data  # дані переживають перезапуск
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d visits"]
      interval: 2s
      timeout: 2s
      retries: 10

volumes:
  db-data:

```

- **Ім'я сервіса як адреса:** app підключається до `host=db` — compose створює мережу, де вбудований DNS резолвить імена сервісів у IP контейнерів. Жодних хардкод-адрес.
- **Том:** `db-data` монтується в `/var/lib/postgresql/data`.
- **Healthcheck:** `pg_isready` кожні 2 с; app має `depends_on: db: condition: service_healthy`, тобто стартує лише коли Postgres реально приймає з'єднання, а не просто «контейнер запущено».

Перевірка (`docs/compose-up.txt` , `docs/compose-test.txt`):

```
$ docker compose up -d --wait
Container domashka-07-db-1 Healthy
Container domashka-07-app-1 Started
$ curl localhost:8007/
Привіт!! Це візит номер 1.
Привіт!! Це візит номер 2.
Привіт!! Це візит номер 3.
```

Том працює: після `docker compose down && up` (контейнери знищено і створено заново) лічильник продовжив з того ж місця — візит номер 5.

Нюанс: у застосунку стартове підключення до бази **навмисно без retry** — якщо база недоступна, контейнер одразу падає. Так поведінка `depends_on` видна неозброєним оком (пункт 4).

4. Зламай й полагодь

4.1. Прибрали healthcheck — що змінилось

Варіант А: healthcheck прибрано, `condition: service_healthy` лишився. Compose навіть не запускає app (docs/broken-A.txt):

```
Container domashka-07-db-1 Error dependency db failed to start
dependency failed to start: container domashka-07-db-1 has no healthcheck configured
```

Умова «чекай healthy» без healthcheck нездійсненна, і compose чесно про це каже. Це «хороша» поломка — помилка миттєва і зрозуміла.

Варіант Б: healthcheck прибрано і `depends_on` спрощено до списку. Це підступніший випадок — `depends_on: [db]` гарантує лише **порядок запуску** контейнерів, а не готовність бази. На чистому томі Postgres кілька секунд виконує initdb, і app програє цю гонку (docs/broken-B.txt):

```
$ docker compose -f compose.broken.yaml ps -a
NAME                ... STATUS
domashka-07-app-1    ... Exited (1) 4 seconds ago
domashka-07-db-1     ... Up 5 seconds

$ docker compose logs app
psycopgp.OperationError: connection failed: connection to server
at "172.22.0.2", port 5432 failed: Connection refused
```

Контейнер бази вже «Started», але сам Postgres усередині ще не слухає порт — app отримав `Connection refused` і впав. Найгірше тут те, що поломка **плаваюча**: на прогітому томі база встигає піднятися швидше за Python-процес, і той самий файл «іноді працює». Класична гонка, яку healthcheck перетворює на детермінований порядок.

Полагодили: повернули healthcheck + `condition: service_healthy` — навіть на повністю чистому томі app стартував після `db ... Healthy` і підключився з першої спроби (`docs/fixed.txt`).

4.2. Опублікували порт бази — і чому так не треба

У зламаній версії додав `ports: "5432:5432"` для db. Технічно все працює — але тепер база видна з хоста і, оскільки bind на `0.0.0.0`, з локальної мережі (`docs/db-port-exposed.txt`):

```
$ nc -vz localhost 5432
Connection to localhost port 5432 [tcp/postgresql] succeeded!

$ psql "host=... user=app password=secret" -c 'SELECT version();'
PostgreSQL 17.10 on aarch64-unknown-linux-musl ...
```

Чому цього робити не варто:

1. **Це не потрібно.** Єдиний клієнт бази — app, і він ходить у `db:5432` через внутрішню мережу compose. Публікація нічого не додає функціонально.
2. **Зайва поверхня атаки.** База з навчальним паролем `secret` стає доступною будь-кому на хості й у Wi-Fi-мережі; сканери ботнетів перебирають відкриті 5432/3306 постійно.
3. **Обхід застосунку.** Уся валідація й логіка живе в app; прямий доступ до бази дозволяє читати і псувати дані повз неї.
4. **Конфлікти портів.** 5432 на хості часто вже зайнятий (локальний Postgres, сусідній проект) — я в цьому переконався ще на кроці 3, коли 8000 виявився зайнятим чужим контейнером.

Якщо доступ ззовні таки потрібен (дебаг), правильні варіанти: `docker compose exec db psql`, тимчасовий bind на `127.0.0.1:5432:5432`, або SSH/VPN-тунель до хоста.

Висновки

1. **Порядок інструкцій у Dockerfile — це і є кеш-стратегія.** Стабільне (залежності) — вище, мінливе (код) — нижче: збірка з 7.66 с перетворюється на 1.09 с, і різниця росте з розміром залежностей.
2. **Екес-форма CMD — про сигнали, а не про стиль.** Без неї PID 1 — це `sh`, який не пересилає SIGTERM, і кожен `docker stop` чекає таймаут.

3. **depends_on без healthcheck — це ілюзія готовності.** Він упорядковує лише запуск контейнерів; готовність сервісу всередині перевіряє тільки healthcheck. Поломка без нього — плаваюча гонка, найнеприємніший клас багів.
4. **Публікувати треба тільки те, що споживають ззовні.** Внутрішній трафік compose ходить мережею проєкту за іменами сервісів; кожен зайвий `ports:` — це мінус безпека і плюс конфлікти.

Файли

- `app/` — `app.py`, `requirements.txt`, `Dockerfile`, `.dockerignore`
 - `compose.yaml` — робоча конфігурація; `compose.broken.yaml` — «зламана» для пункту 4
 - `docs/` — повні логи: збірки (`build-1.txt`, `build-2.txt`), `compose` (`compose-up.txt`, `compose-test.txt`), поломки (`broken-A.txt`, `broken-B.txt`, `db-port-exposed.txt`), лагодження (`fixed.txt`), образ (`image-info.txt`)
-